CORPORACIÓN UNIFICADA NACIONAL DE EDUCACIÓN SUPERIOR — CUN

Programa: Ingeniería de Sistemas
Asignatura: Programación Avanzada

SISTEMA DE GESTIÓN DE BIBLIOTECA

Documentación técnica — ACA Final 2026

Estudiante: Marlon Fabián Moncaleano Núñez
Tecnologías: C# · Windows Forms (.NET 8) · SQL Server
Fecha: Noviembre de 2026

Tabla de contenido

1. Introducción
2. Objetivos
3. Planteamiento del problema
4. Análisis de requerimientos
5. Casos de uso
6. Diagrama de clases
7. Modelo entidad-relación
8. Diccionario de datos
9. Arquitectura del sistema
10. Explicación de los módulos desarrollados
11. Descripción de la conexión con SQL Server
12. Validaciones y manejo de excepciones
13. Capturas de pantalla del sistema
14. Pruebas de funcionamiento
15. Control de versiones con Git y GitHub
16. Conclusiones
17. Recomendaciones
18. Referencias bibliográficas

> **Numeración de páginas:** el formato Markdown no tiene paginación. La versión `DOCUMENTACION.pdf` incluye la misma información con número de página en el pie.

1. Introducción

Las instituciones educativas manejan un volumen creciente de material bibliográfico y de usuarios que lo consultan. Cuando el control de libros, autores, editoriales y préstamos se hace en hojas de cálculo o de forma manual, aparecen duplicados, existencias que no coinciden con la realidad y préstamos sin trazabilidad.

Este documento describe el diseño, la implementación y las pruebas del **Sistema de Gestión de Biblioteca**, una aplicación de escritorio desarrollada con **C#, Windows Forms y SQL Server**. El sistema permite administrar libros, autores, editoriales y usuarios (operaciones de registro, consulta, actualización y eliminación), controlar préstamos y devoluciones con actualización automática de existencias, y consultar reportes.

El código aplica los principios de la **Programación Orientada a Objetos** y una **arquitectura por capas** (Presentación, Lógica de negocio, Acceso a datos y Base de datos), con consultas parametrizadas, validación de datos y manejo de excepciones.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de escritorio para la gestión integral de una biblioteca utilizando C#, Windows Forms y SQL Server, que permita administrar libros, autores, editoriales y usuarios, gestionar préstamos y devoluciones, y generar reportes mediante operaciones conectadas a una base de datos relacional, aplicando los principios de la Programación Orientada a Objetos.

2.2 Objetivos específicos

- Aplicar los principios de Programación Orientada a Objetos (clases, objetos, constructores, encapsulamiento, herencia y polimorfismo).
- Diseñar una base de datos relacional normalizada con claves primarias, foráneas y restricciones de integridad.
- Implementar operaciones CRUD para libros, autores, editoriales, usuarios y préstamos.
- Utilizar SQL Server como gestor de base de datos, incluyendo vistas para los reportes.
- Implementar una arquitectura por capas con separación física en carpetas y espacios de nombres.
- Validar la información ingresada por el usuario y manejar excepciones con mensajes claros.
- Utilizar Git y GitHub como sistema de control de versiones.
- Documentar técnicamente el proyecto.
- Implementar formularios que permitan administrar la información de libros, autores, editoriales y usuarios.

3. Planteamiento del problema

Una institución educativa requiere una aplicación de escritorio que facilite la administración de los procesos de su biblioteca. La gestión de libros, autores, editoriales y usuarios, así como el control de préstamos y devoluciones, necesita un sistema organizado que permita registrar, consultar, actualizar y eliminar información, hacer seguimiento al estado de los préstamos y generar reportes.

Sin un sistema centralizado se presentan los siguientes problemas:

- **Inconsistencia de inventario:** no se sabe cuántos ejemplares hay realmente disponibles.
- **Datos duplicados o erróneos:** usuarios o libros registrados varias veces, ISBN repetidos, correos inválidos.
- **Falta de trazabilidad:** no se conoce quién tiene un libro, desde cuándo, ni el historial de préstamos.
- **Dificultad para consultar:** obtener un listado consolidado exige revisar archivos manualmente.

La solución propuesta es un sistema de escritorio que centraliza la información en SQL Server, valida los datos antes de almacenarlos y garantiza la integridad entre libros, usuarios y préstamos.

4. Análisis de requerimientos

4.1 Requerimientos funcionales

Código	Requerimiento	Implementación

RF01	**Gestión de libros:** registrar, listar, editar, eliminar y buscar (código/ISBN, título, autor); relacionar con autor y editorial; consultar existencias	`FormLibros`, `LibroBLL`, `LibroDAL`
RF02	**Gestión de autores:** registrar, listar, editar y eliminar (si no tienen libros)	`FormAutores`, `AutorBLL`, `AutorDAL`
RF03	**Gestión de editoriales:** registrar, listar, editar y eliminar (si no tienen libros)	`FormEditoriales`, `EditorialBLL`, `EditorialDAL`
RF04	**Gestión de usuarios:** registrar, listar, editar y eliminar (si no tienen préstamos)	`FormUsuarios`, `UsuarioBLL`, `UsuarioDAL`
RF05	**Gestión de préstamos:** registrar préstamo (usuario + libro + fecha), consultar, editar fechas, eliminar y filtrar por estado	`FormPrestamos`, `PrestamoBLL`, `PrestamoDAL`
RF06	**Gestión de devoluciones:** registrar devolución, actualizar fecha, estado «Devuelto» y existencias del libro	`PrestamoDAL.RegistrarDevolucion` (transacción)
RF07	**Consultas y reportes:** 9 reportes basados en vistas SQL, con exportación a CSV	`FormReportes`, `ReporteBLL`, `ReporteDAL`
RF08	**Navegación:** formulario principal con acceso a Inicio, Libros, Usuarios, Autores, Editoriales, Préstamos y Reportes	`FormPrincipal`

4.2 Requerimientos no funcionales

Requerimiento	Cómo se cumple
Interfaz clara y organizada	Menú lateral, formularios con la misma distribución (datos a la izquierda, búsqueda y tabla a la derecha)
Validación de campos obligatorios	`Validador` en la capa de negocio y restricciones `NOT NULL` en la base de datos
Prevención de duplicados	Restricciones `UNIQUE` (ISBN, documento, correos, nombre de editorial) con mensajes claros
Mensajes claros	Clase `Mensajes`; las excepciones SQL se traducen en `ConexionBD.Traducir`
Manejo adecuado de excepciones	`ReglaNegocioException` para errores esperados; `FormCrudBase.Ejecutar` captura todo y evita que la aplicación se cierre
Código organizado y legible	Carpetas por capa, espacios de nombres por capa, nombres descriptivos
Integridad de datos	PK, FK, `CHECK`, transacciones en préstamos y devoluciones
Conexión con SQL Server	`ConexionBD` + cadena en `App.config`
Consultas parametrizadas	Todas las consultas usan `SqlParameter`; los filtros `LIKE` escapan los comodines
Separación de responsabilidades	Los formularios no contienen SQL; el acceso a datos no contiene reglas de interfaz

4.3 Reglas de negocio implementadas

1. Un libro no se puede prestar si no tiene existencias disponibles.
2. Un usuario puede tener como máximo **3 préstamos activos** simultáneos.
3. Un usuario no puede tener dos préstamos activos del mismo libro.
4. Al registrar un préstamo se descuenta una unidad del stock; al devolver se repone. Ambas acciones ocurren en una **transacción**.
5. Un préstamo ya devuelto no se puede devolver de nuevo (evita inflar el stock).
6. La fecha de devolución no puede ser anterior a la del préstamo ni futura; un préstamo «Prestado» no tiene fecha de devolución y uno «Devuelto» siempre la tiene (también garantizado por la restricción `CK_Prestamos_Coherencia`).
7. Al eliminar un préstamo activo se repone el stock del libro.
8. No se puede eliminar un libro con préstamos activos ni con historial, ni un autor/editorial con libros, ni un usuario con préstamos.
9. El campo **Stock** representa los ejemplares **disponibles** en este momento (ya descuenta los prestados).

5. Casos de uso

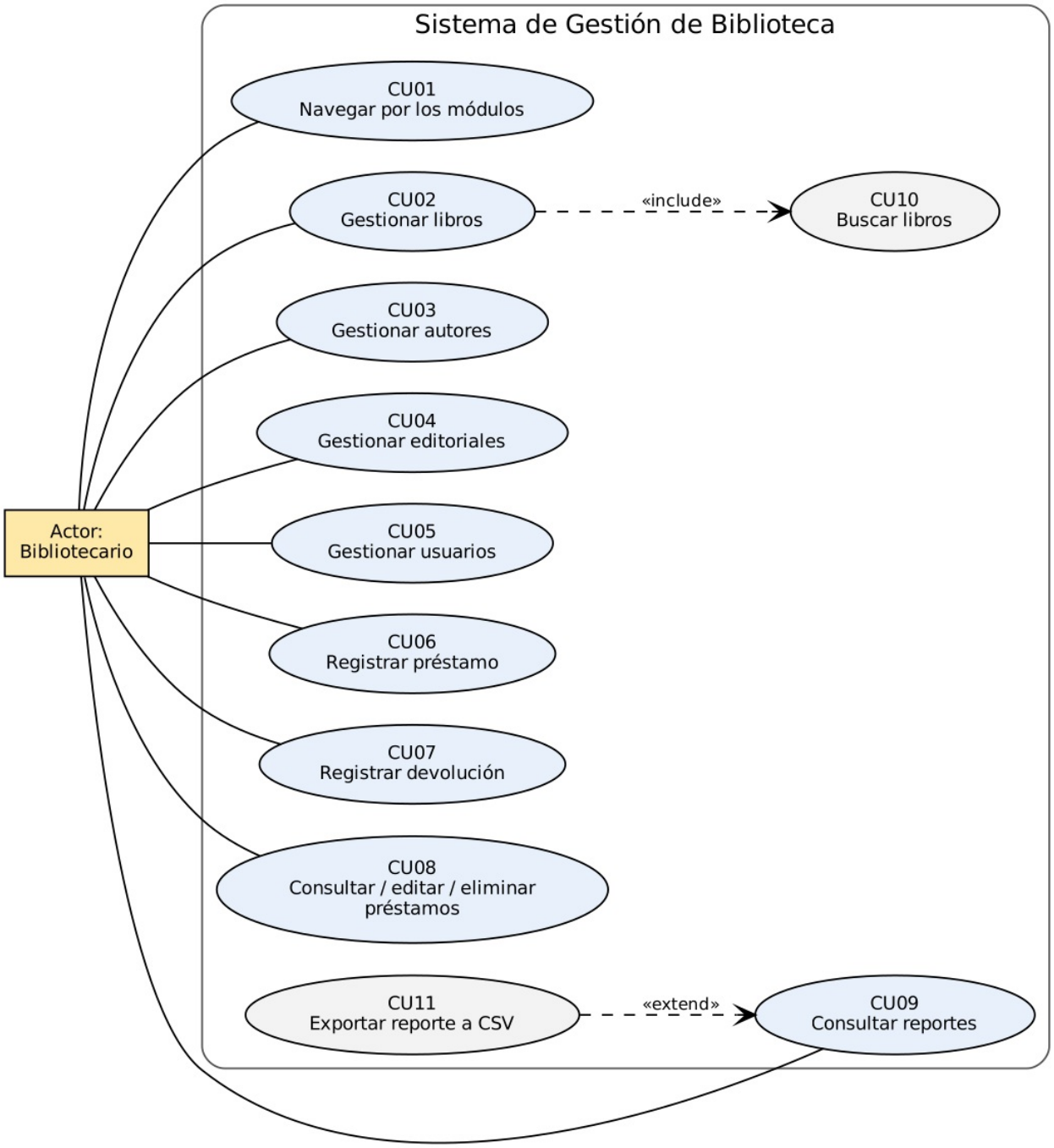
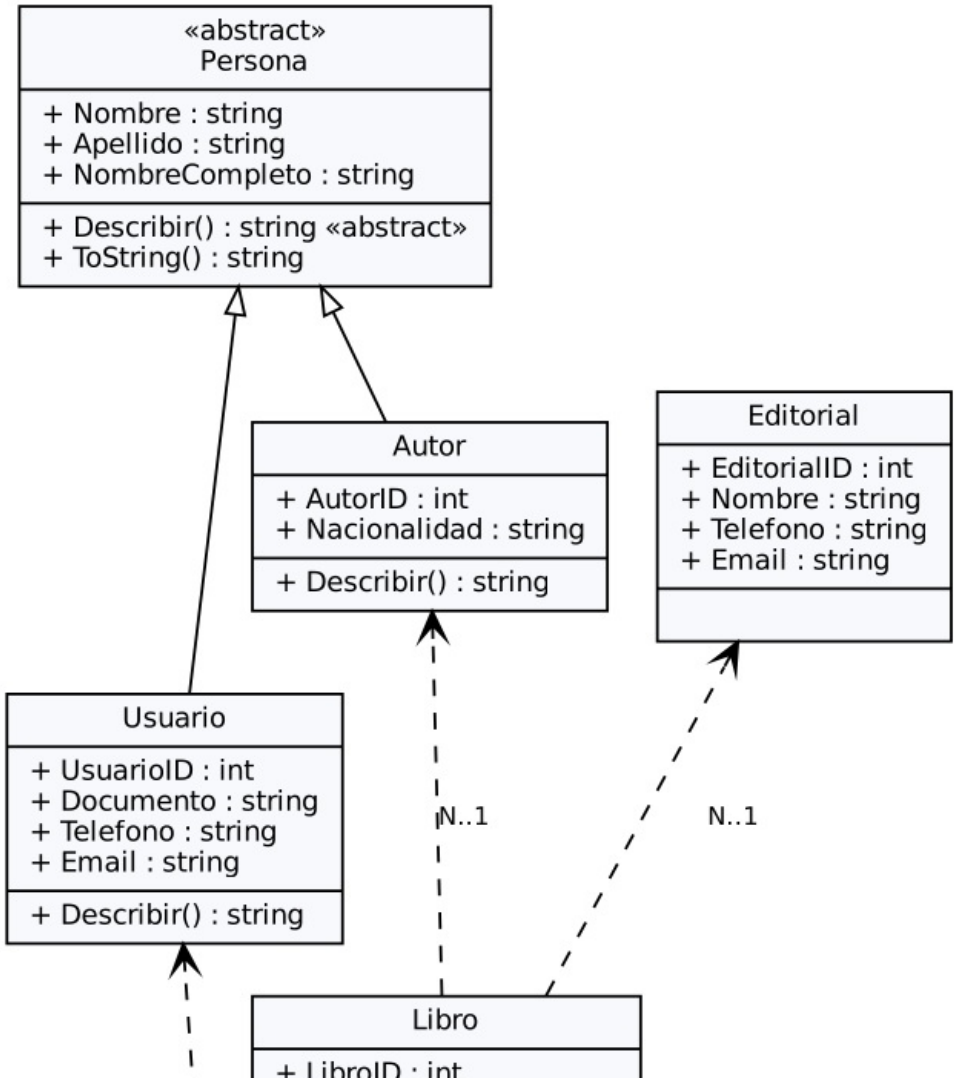


Diagrama de casos de uso

Caso de uso	Actor	Flujo principal	Flujos alternos
CU01 Navegar por los módulos	Bibliotecario	Selecciona un módulo en el menú lateral y el sistema carga su formulario	Si no hay conexión con SQL Server se muestra una advertencia
CU02 Gestionar libros	Bibliotecario	Completa ISBN, título, autor, editorial, año y existencias → Guardar. Selecciona una fila para Actualizar o Eliminar	ISBN duplicado, autor inexistente, año fuera de rango o libro con préstamos: se muestra el motivo y no se modifica nada
CU03 Gestionar autores	Bibliotecario	Registra nombre, apellido y nacionalidad; edita o elimina	No se elimina un autor con libros

CU04 Gestionar editoriales	Bibliotecario	Registra nombre, teléfono y correo; edita o elimina	Nombre o correo duplicado; editorial con libros
CU05 Gestionar usuarios	Bibliotecario	Registra documento, nombre, apellido, teléfono y correo; edita o elimina	Documento o correo duplicado; correo inválido; usuario con préstamos
CU06 Registrar préstamo	Bibliotecario	Elige usuario, libro y fecha → «Registrar préstamo»; el sistema valida, registra y descuenta stock	Sin existencias, máximo de préstamos alcanzado, libro ya prestado al usuario
CU07 Registrar devolución	Bibliotecario	Selecciona un préstamo activo → «Registrar devolución»; se guarda la fecha, el estado pasa a «Devuelto» y el stock aumenta	El préstamo ya estaba devuelto
CU08 Consultar, editar y eliminar préstamos	Bibliotecario	Filtra por estado o texto; edita fechas o elimina	Estado y fechas incoherentes se rechazan
CU09 Consultar reportes	Bibliotecario	Elige un reporte y el sistema lo muestra en una tabla	Reporte vacío: se muestra 0 registros
CU10 Buscar libros	Bibliotecario	Elige criterio (Todos, ISBN, Título, Autor), escribe el texto y busca	Sin coincidencias: tabla vacía
CU11 Exportar reporte a CSV	Bibliotecario	Pulsa «Exportar a CSV» y elige la ruta del archivo	Sin datos o error de escritura: se informa al usuario

6. Diagrama de clases



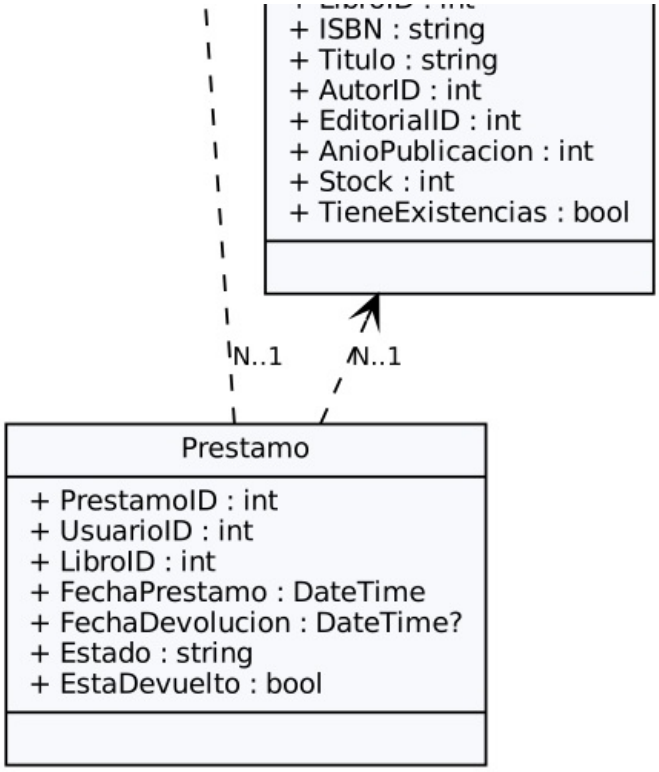
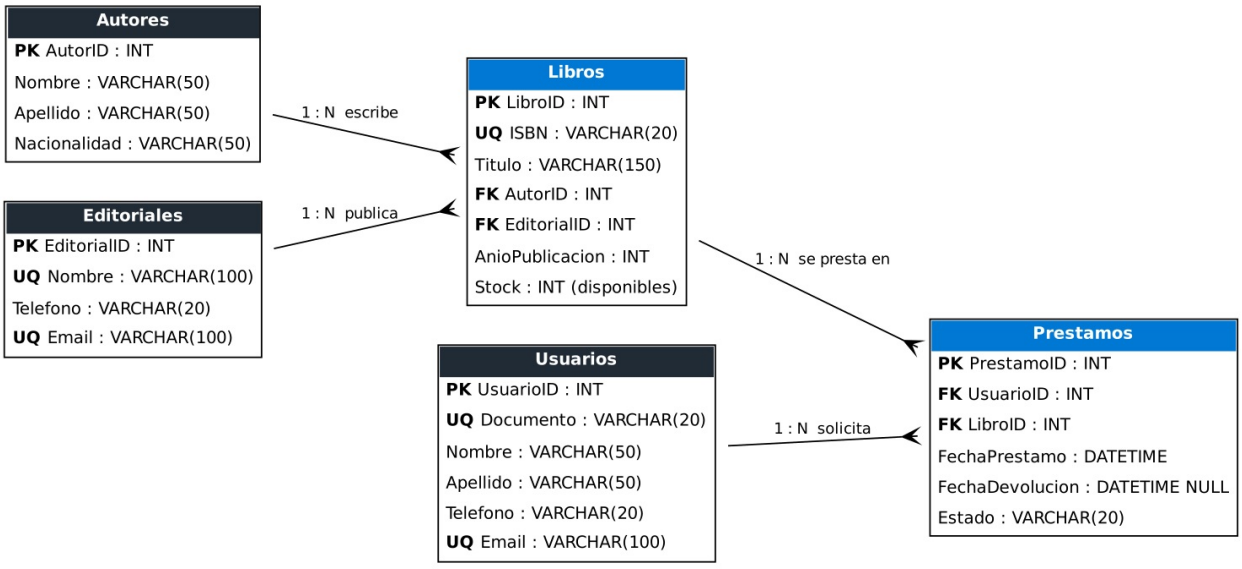


Diagrama de clases de las entidades

Aplicación de POO en el código:

Concepto	Dónde se evidencia
Clases y objetos	`Autor`, `Usuario`, `Editorial`, `Libro`, `Prestamo`, DAL, BLL y formularios
Constructores	Constructor vacío y constructor con parámetros en cada entidad; constructores de los formularios que construyen la interfaz
Encapsulamiento y propiedades	Propiedades automáticas en las entidades; campos privados `readonly` en DAL/BLL/formularios; propiedades calculadas como `NombreCompleto`, `TieneExistencias` y `EstaDevuelto`
Herencia	`Autor` y `Usuario` heredan de `Persona` (comparten nombre y apellido). `FormLibros`, `FormAutores`, `FormEditoriales`, `FormUsuarios` y `FormPrestamos` heredan de `FormCrudBase`
Polimorfismo	`Persona.Describir()` es abstracto y cada subclase lo especializa. `FormCrudBase` declara métodos abstractos (`Guardar`, `Actualizar`, `Eliminar`, `Refrescar`…) que cada módulo implementa, y la clase base los invoca sin conocer el módulo concreto
Colecciones	`Dictionary<string,string>` con los mensajes de restricciones, arreglos de criterios/estados, `DataTable` para listados
Organización de responsabilidades	Entidades (datos), DAL (SQL), BLL (reglas), formularios (interfaz)

7. Modelo entidad-relación



Modelo entidad-relación

Relaciones:

- Un **autor** escribe muchos **libros**; cada libro tiene un autor (1:N).
- Una **editorial** publica muchos **libros**; cada libro tiene una editorial (1:N).
- Un **usuario** puede tener muchos **préstamos**; cada préstamo pertenece a un usuario (1:N).
- Un **libro** puede aparecer en muchos **préstamos**; cada préstamo es de un libro (1:N).

Las relaciones entre Usuarios y Libros (N:M) se resuelven con la tabla **Prestamos**, que además guarda las fechas y el estado. No se necesita una tabla `DetallePrestamos` porque cada préstamo corresponde a un solo libro.

8. Diccionario de datos

Base de datos: `BibliotecaDB` (SQL Server). Los nombres de las restricciones coinciden con los del script `sql/Script_Biblioteca_Completo.sql`.

8.1 Autores

Campo	Tipo	Nulo	Restricción	Descripción
`AutorID`	INT IDENTITY(1,1)	No	PK `PK_Autores`	Identificador único del autor
`Nombre`	VARCHAR(50)	No		Nombre del autor
`Apellido`	VARCHAR(50)	No		Apellido del autor
`Nacionalidad`	VARCHAR(50)	No		País de origen

8.2 Editoriales

Campo	Tipo	Nulo	Restricción	Descripción
`EditorialID`	INT IDENTITY(1,1)	No	PK `PK_Editoriales`	Identificador único
`Nombre`	VARCHAR(100)	No	UNIQUE `UQ_Editoriales_Nombre`	Nombre de la editorial
`Telefono`	VARCHAR(20)	No		Teléfono de contacto
`Email`	VARCHAR(100)	No	UNIQUE `UQ_Editoriales_Email`; CHECK `CK_Editoriales_Email` (formato)	Correo de contacto

8.3 Libros

Campo	Tipo	Nulo	Restricción	Descripción
`LibroID`	INT IDENTITY(1,1)	No	PK `PK_Libros`	Identificador único
`ISBN`	VARCHAR(20)	No	UNIQUE `UQ_Libros_ISBN`	Código del libro
`Titulo`	VARCHAR(150)	No	Índice `IX_Libros_Titulo`	Título
`AutorID`	INT	No	FK `FK_Libros_Autores` → Autores	Autor del libro
`EditorialID`	INT	No	FK `FK_Libros_Editoriales` → Editoriales	Editorial del libro
`AnioPublicacion`	INT	No	CHECK `CK_Libros_Anio` (1001–2100); la aplicación limita al año actual	Año de publicación
`Stock`	INT	No	DEFAULT 1; CHECK `CK_Libros_Stock` (≥ 0)	Ejemplares **disponibles**

8.4 Usuarios

Campo	Tipo	Nulo	Restricción	Descripción
`UsuarioID`	INT IDENTITY(1,1)	No	PK `PK_Usuarios`	Identificador único
`Documento`	VARCHAR(20)	No	UNIQUE `UQ_Usuarios_Documento`	Cédula / identificación
`Nombre`	VARCHAR(50)	No		Nombre
`Apellido`	VARCHAR(50)	No		Apellido
`Telefono`	VARCHAR(20)	No		Teléfono
`Email`	VARCHAR(100)	No	UNIQUE `UQ_Usuarios_Email`; CHECK `CK_Usuarios_Email` (formato)	Correo electrónico

8.5 Prestamos

Campo	Tipo	Nulo	Restricción	Descripción
`PrestamoID`	INT IDENTITY(1,1)	No	PK `PK_Prestamos`	Identificador único
`UsuarioID`	INT	No	FK `FK_Prestamos_Usuarios` → Usuarios	Usuario que recibe el libro
`LibroID`	INT	No	FK `FK_Prestamos_Libros` → Libros	Libro prestado
`FechaPrestamo`	DATETIME	No	DEFAULT GETDATE()	Fecha y hora del préstamo
`FechaDevolucion`	DATETIME	Sí		Fecha y hora de devolución (nula mientras esté prestado)
`Estado`	VARCHAR(20)	No	DEFAULT ‘Prestado’; CHECK `CK_Prestamos_Estado` (‘Prestado’, ‘Devuelto’)	Estado del préstamo

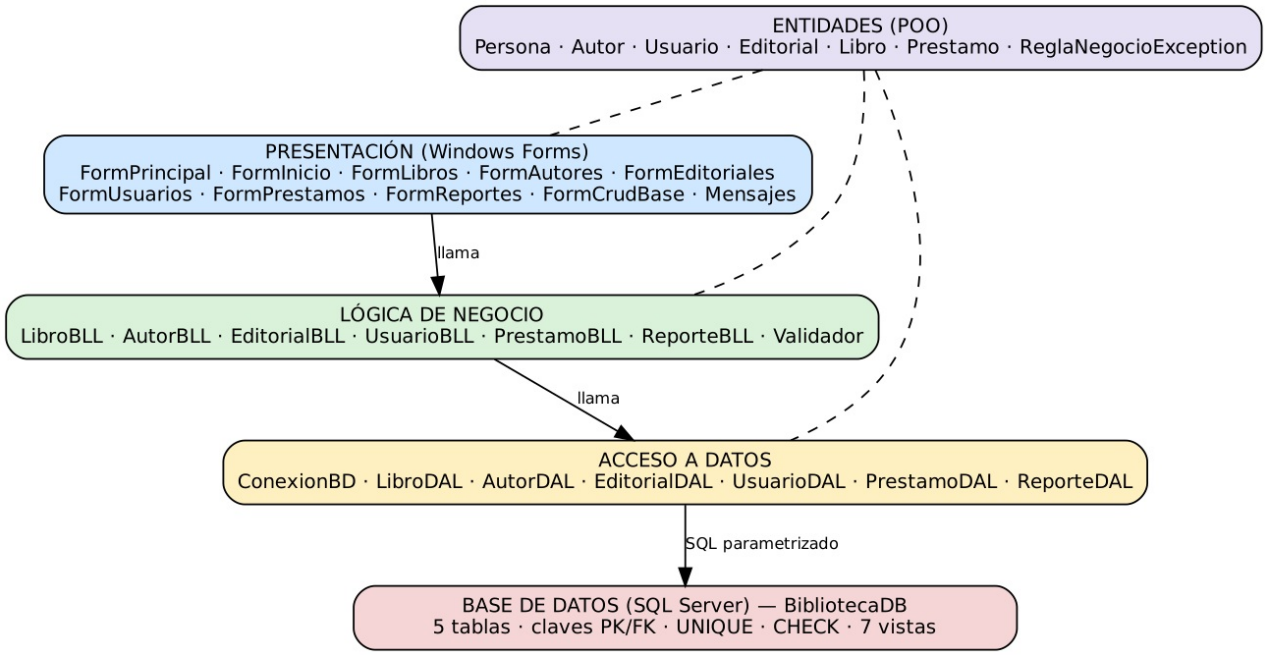
Restricción adicional `CK_Prestamos_Coherencia`: `Prestado` exige `FechaDevolucion` nula; `Devuelto` exige fecha de devolución mayor o igual a la fecha de préstamo.

8.6 Vistas de reporte

Vista	Contenido
`VISTA_InventarioLibros`	Libros con autor, editorial, año y ejemplares disponibles
`VISTA_ResumenAutores`	Autores con cantidad de libros registrados
`VISTA_ResumenEditoriales`	Editoriales con cantidad de libros registrados

`VISTA_PrestamosActivos`	Préstamos en estado «Prestado» con usuario, libro y autor
`VISTA_HistorialPrestamos`	Todos los préstamos con usuario, libro, fechas y estado
`VISTA_LibrosMasPrestados`	Cantidad total de préstamos por libro
`VISTA_ResumenGeneral`	Indicadores consolidados (títulos, ejemplares, autores, editoriales, usuarios, préstamos activos y devueltos)

9. Arquitectura del sistema



Arquitectura por capas

La separación en capas es **física** (carpetas y espacios de nombres distintos dentro del proyecto de Visual Studio) y **lógica** (cada capa solo conoce a la inferior).

```
BibliotecaApp/
  Entidades/       BibliotecaApp.Entidades      Persona, Autor, Usuario, Editorial, Libro, Prestamo, ReglaNegocioException
  AccesoDatos/     BibliotecaApp.AccesoDatos    ConexionBD, AutorDAL, EditorialDAL, LibroDAL, UsuarioDAL, PrestamoDAL, ReporteDAL
  LogicaNegocio/   BibliotecaApp.LogicaNegocio  Validador, AutorBLL, EditorialBLL, LibroBLL, UsuarioBLL, PrestamoBLL, ReporteBLL
  Presentacion/    BibliotecaApp.Presentacion   FormPrincipal, FormInicio, FormCrudBase, FormLibros, FormAutores,
                                                FormEditoriales, FormUsuarios, FormPrestamos, FormReportes, Mensajes
```

Capa	Responsabilidad
Presentación	Formularios y controles, captura de datos, mensajes al usuario. No contiene SQL
Lógica de negocio	Validación de datos, reglas del sistema (máximo de préstamos, duplicados activos, rangos) y coordinación de las operaciones
Acceso a datos	Conexión con SQL Server, consultas parametrizadas, transacciones y traducción de errores SQL
Base de datos	Tablas, relaciones, restricciones de integridad y vistas de reportes
Entidades	Modelo de objetos compartido por las capas

Flujo de una operación (ejemplo: registrar préstamo): `FormPrestamos.Guardar()` → `PrestamoBLL.RegistrarPrestamo()` valida reglas → `PrestamoDAL.RegistrarPrestamo()` abre una transacción, verifica usuario y libro, bloquea la fila del libro, inserta el préstamo y descuenta el stock → `Commit`. Ante cualquier error se hace `Rollback` y la excepción sube hasta `FormCrudBase.Ejecutar`, que muestra el mensaje.

10. Explicación de los módulos desarrollados

10.1 Inicio (`FormInicio`)

Pantalla de bienvenida con tarjetas de indicadores tomadas de `VISTA_ResumenGeneral`.

10.2 Formulario principal y navegación (`FormPrincipal`)

Menú lateral con los siete módulos, encabezado con el nombre del módulo activo y barra de estado que informa si la conexión con SQL Server fue exitosa. Cada módulo se crea al seleccionarlo y se libera al cambiar, de modo que las listas desplegables siempre muestran datos actualizados.

10.3 Libros (`FormLibros`)

Campos: ISBN, título, autor y editorial (listas desplegables cargadas desde la base de datos), año de publicación y ejemplares disponibles. Búsqueda por **Todos, Código (ISBN), Título o Autor**. Valida la unicidad del ISBN, que autor y editorial existan, el rango del año y las existencias. No permite eliminar libros con préstamos.

10.4 Autores (`FormAutores`) y Editoriales (`FormEditoriales`)

CRUD con búsqueda. Los nombres de personas solo admiten letras, espacios, apóstrofes, puntos y guiones. En editoriales se valida teléfono, correo y unicidad de nombre y correo. La eliminación se bloquea con un mensaje claro si hay libros relacionados.

10.5 Usuarios (`FormUsuarios`)

CRUD con búsqueda por documento, nombre, apellido o correo. El documento debe tener solo dígitos (5 a 20) y ser único; el correo se valida por formato y unicidad.

10.6 Préstamos y devoluciones (`FormPrestamos`)

- **Registrar préstamo:** se elige usuario, libro y fecha; el estado inicia en «Prestado» y el stock baja en uno.
- **Registrar devolución:** se selecciona un préstamo activo; se guarda fecha y hora, el estado pasa a «Devuelto» y el stock sube en uno.
- **Actualizar fechas:** se corrigen las fechas manteniendo la coherencia con el estado. El usuario y el libro de un préstamo existente no se modifican para no descuadrar el inventario; si hay un error se elimina y se registra de nuevo.
- **Eliminar:** si el préstamo estaba activo, la unidad vuelve al stock.
- **Filtro** por estado (Todos, Prestado, Devuelto) y búsqueda por documento, nombre, título o ISBN.

10.7 Reportes (`FormReportes`)

Nueve reportes: resumen consolidado, libros, autores, editoriales, usuarios, préstamos activos, préstamos devueltos, historial y libros más prestados. Incluye **exportación a CSV** (UTF-8 con BOM, separador `;`, compatible con Excel).

10.8 Clase base `FormCrudBase`

Construye la interfaz común de los módulos CRUD y centraliza: manejo de excepciones, habilitación de botones (Guardar solo para registros nuevos; Actualizar y Eliminar solo con un registro seleccionado), confirmación antes de eliminar y refresco de la tabla.

11. Descripción de la conexión con SQL Server

La clase `ConexionBD` (capa de acceso a datos) es el único punto que conoce la cadena de conexión, que se lee de `App.config`:

```
        <connectionStrings>
          <add name="BibliotecaDB"
               connectionString="Server=localhost;Database=BibliotecaDB;Integrated Security=True;TrustServerCertificate=True;Connect
Timeout=10;"
               providerName="Microsoft.Data.SqlClient" />
        </connectionStrings>
```

- Se usa el paquete **Microsoft.Data.SqlClient** (el proveedor actual de Microsoft para SQL Server).
- `ConexionBD` ofrece `Consultar`, `Ejecutar` y `Escalar`, que abren y cierran la conexión con `using` y siempre reciben parámetros (`SqlParameter`), lo que previene inyección SQL. Los textos de búsqueda se escapan con `PatronLike` para que `%`, `_` y `[` se

traten como texto.

- `PrestamoDAL` usa `SqlTransaction` para que el préstamo (o la devolución) y el cambio de stock se apliquen juntos o no se apliquen. La lectura del libro usa `UPDLOCK` para evitar que dos usuarios consuman la última existencia al mismo tiempo.
- `ConexionBD.Traducir` convierte los errores de SQL Server en mensajes comprensibles: errores de conexión (números 2, 53, 4060, 18456…), duplicados (2627/2601) y violaciones de claves foráneas o `CHECK` (547), identificando la restricción por su nombre.
- `ProbarConexion` se ejecuta al abrir la aplicación y muestra el resultado en la barra de estado.

Para ejecutar el sistema: ejecutar `sql/Script_Biblioteca_Completo.sql` en SSMS y ajustar `Server` en `App.config` (`localhost`, `.\SQLEXPRESS` o `(localdb)\MSSQLLocalDB`).

12. Validaciones y manejo de excepciones

Entidad	Validaciones (capa de negocio)	Refuerzo en la base de datos
Libros	ISBN obligatorio, 10–20 caracteres (dígitos, guiones, X final); título obligatorio ≤ 150; autor y editorial seleccionados y existentes; año entre 1001 y el año actual; existencias entre 0 y 100000	`UNIQUE` en ISBN, `CHECK` de año y stock, FK
Autores	Nombre, apellido y nacionalidad obligatorios, ≤ 50, solo letras	`NOT NULL`
Editoriales	Nombre ≤ 100; teléfono de 7 a 20 dígitos; correo válido ≤ 100	`UNIQUE` en nombre y correo, `CHECK` de correo
Usuarios	Documento numérico 5–20; nombre y apellido solo letras; teléfono 7–20 dígitos; correo válido	`UNIQUE` en documento y correo, `CHECK` de correo
Préstamos	Usuario y libro seleccionados y existentes; fechas no futuras; devolución ≥ préstamo; existencias; máximo de préstamos activos; sin duplicar libro activo	FK, `CHECK` de estado y coherencia, transacciones

Manejo de excepciones:

- `ReglaNegocioException` representa errores esperados (validación, regla incumplida, duplicado, registro relacionado). Se muestra como **advertencia**.
- Cualquier otra excepción (por ejemplo, falla de conexión) se muestra como **error** con un mensaje explicativo.
- `FormCrudBase.Ejecutar` envuelve cada acción del usuario, y `Program.cs` registra un manejador global como último recurso, de modo que la aplicación no se cierra por un error.
- Los métodos transaccionales siempre ejecutan `Rollback` ante cualquier fallo.

13. Capturas de pantalla del sistema

> **Importante:** las imágenes actuales de `docs/capturas/` son marcadores de posición. Ejecute el sistema, tome las capturas y reemplace cada archivo conservando el mismo nombre; el documento y el PDF se actualizan automáticamente.

Inicio



Libros



Autores



Editoriales



Usuarios



Préstamos



Devolución registrada



Reportes



Mensaje de validación



Base de datos en SQL Server Management Studio



14. Pruebas de funcionamiento

14.1 Pruebas automáticas (proyecto `BibliotecaApp.Pruebas`)

Se ejecutan con `dotnet run --project BibliotecaApp.Pruebas` y no requieren base de datos. Resultado de la última ejecución: **19 de**

19 pruebas aprobadas.

N.º	Prueba	Resultado
1	Nombre válido con tilde y espacios se acepta y se limpia	Aprobada
2	Nombre con dígitos es rechazado	Aprobada
3	Campo obligatorio vacío es rechazado	Aprobada
4	Longitud máxima excedida es rechazada	Aprobada
5	Correo válido se acepta	Aprobada
6	Correo sin dominio válido es rechazado	Aprobada
7	Teléfono válido (se quitan espacios)	Aprobada
8	Teléfono demasiado corto es rechazado	Aprobada
9	Documento numérico válido	Aprobada
10	Documento con letras es rechazado	Aprobada
11	ISBN del script de datos es válido	Aprobada
12	ISBN con X final se normaliza a mayúscula	Aprobada
13	ISBN corto es rechazado	Aprobada
14	Entero fuera de rango es rechazado	Aprobada
15	Lista desplegable sin selección es rechazada	Aprobada
16	Polimorfismo: `Autor.Describir()`	Aprobada
17	Polimorfismo: `Usuario.ToString()`	Aprobada
18	Libro con stock 0 no tiene existencias	Aprobada
19	Un préstamo nuevo inicia en estado «Prestado»	Aprobada

14.2 Pruebas funcionales con la aplicación y la base de datos

Estos casos requieren SQL Server. Ejecútelos con los datos del script y complete la columna **Resultado obtenido** (Aprobada / Fallida) con una captura cuando sea relevante.

ID	Módulo	Caso de prueba	Datos de entrada	Resultado esperado	Resultado obtenido
PF01	Autores	Registrar autor válido	Pablo / Neruda / Chilena	Mensaje de éxito; aparece en la tabla	
PF02	Autores	Campo obligatorio vacío	Nombre vacío	Advertencia «El campo ‘Nombre’ es obligatorio»	
PF03	Autores	Eliminar autor con libros	Gabriel García Márquez	Advertencia: autor relacionado con libros	
PF04	Editoriales	Correo duplicado	Correo `atencion@planeta.com` (ya registrado en Grupo Editorial Planeta)	Advertencia de correo duplicado	
PF05	Usuarios	Documento duplicado	Documento `1018456789`	Advertencia de documento duplicado	
PF06	Usuarios	Correo inválido	`correo@dominio`	Advertencia de formato de correo	
PF07	Usuarios	Eliminar usuario con préstamos	Marlon Moncaleano	Advertencia: usuario con préstamos	
PF08	Libros	Registrar libro válido	ISBN `978-958-00-0000-0`, autor y editorial existentes, año 2020, 3 ejemplares	Mensaje de éxito	
PF09	Libros	ISBN duplicado	ISBN `978-958-42-2000-4`	Advertencia «Ya existe un libro	

				registrado con ese ISBN»	
PF10	Libros	Año futuro	Año mayor al actual	El control numérico no lo permite; la capa de negocio lo rechaza	
PF11	Libros	Buscar por título	Criterio Título, texto `soledad`	Se muestra «Cien años de soledad»	
PF12	Libros	Buscar por autor	Criterio Autor, texto `Borges`	Se muestra «Ficciones»	
PF13	Libros	Eliminar libro con préstamo activo	«Ficciones»	Advertencia: tiene préstamos activos	
PF14	Préstamos	Registrar préstamo	Usuario 6 (Valentina Rojas), libro «Delirio»	Préstamo «Prestado»; stock de Delirio baja de 4 a 3	
PF15	Préstamos	Libro sin existencias	Reducir a 0 el stock de un libro y prestarlo	Advertencia: sin existencias disponibles	
PF16	Préstamos	Máximo de préstamos	Registrar 3 préstamos activos a un mismo usuario y solicitar un cuarto	Advertencia de máximo de 3 préstamos	
PF17	Préstamos	Libro repetido	Usuario 1 pide «Ficciones» otra vez	Advertencia: ya tiene prestado ese libro	
PF18	Devoluciones	Registrar devolución	Seleccionar préstamo activo → Registrar devolución	Estado «Devuelto», fecha asignada, stock +1	
PF19	Devoluciones	Devolver dos veces	Intentar devolver un préstamo ya devuelto	Botón deshabilitado; si se fuerza, advertencia	
PF20	Préstamos	Eliminar préstamo activo	Eliminar un préstamo «Prestado»	Préstamo eliminado y stock +1	
PF21	Reportes	Resumen consolidado	Seleccionar el reporte	Totales coherentes con las tablas	
PF22	Reportes	Exportar CSV	Exportar «Historial de préstamos»	Archivo `.csv` que abre correctamente en Excel	
PF23	Conexión	SQL Server apagado	Detener el servicio y abrir la aplicación	Advertencia clara y estado «Sin conexión»	

15. Control de versiones con Git y GitHub

El proyecto se gestiona con Git y se publica en GitHub. El repositorio contiene el código fuente (`BibliotecaApp/`), las pruebas, el script SQL (`sql/`), el `README.md` con instrucciones de ejecución, la documentación, los diagramas y las capturas de pantalla (`docs/`).

La guía `GUIA_GIT.md` describe el flujo de trabajo y una secuencia de commits descriptivos por avance. El historial se evidencia con `git log --oneline --graph` y con la pestaña *Commits* del repositorio en GitHub.

Enlace del repositorio: `https://github.com/<tu-usuario>/SistemaBiblioteca` *(reemplazar por el enlace real)*

16. Conclusiones

- La arquitectura por capas permitió que los formularios no contengan SQL y que las reglas de negocio se prueben sin base de datos, lo que facilitó el mantenimiento y las pruebas automáticas.
- Las **transacciones** en préstamos y devoluciones garantizan que el inventario nunca quede inconsistente, incluso si ocurre un error a mitad de la operación.
- Combinar validaciones en la capa de negocio con restricciones en SQL Server (`UNIQUE`, `CHECK`, claves foráneas) da doble protección de la integridad: la aplicación informa con mensajes claros y la base de datos impide datos inválidos aunque se acceda por otra vía.

- La herencia (`Persona`, `FormCrudBase`) y el polimorfismo redujeron el código repetido: cinco módulos comparten una misma estructura de interfaz y manejo de errores.
- Las consultas parametrizadas eliminan el riesgo de inyección SQL.

17. Recomendaciones

- Agregar **autenticación y roles** (administrador y bibliotecario) con contraseñas almacenadas con hash.
- Implementar **fecha límite de devolución, multas y renovaciones** de préstamos.
- Permitir **varios libros por préstamo** mediante una tabla `DetallePrestamos`.
- Generar reportes en **PDF o Excel** con gráficos.
- Añadir **pruebas de integración** con una base de datos de prueba y pruebas de interfaz.
- Mover la cadena de conexión a variables de entorno o a un almacén seguro de secretos al desplegar en producción.
- Hacer copias de seguridad periódicas de `BibliotecaDB`.

18. Referencias bibliográficas

Date, C. J. (2019). *Introducción a los sistemas de bases de datos* (8.ª ed.). Pearson Educación.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Microsoft. (2024a). *C# documentation*. Microsoft Learn. https://learn.microsoft.com/dotnet/csharp/

Microsoft. (2024b). *Windows Forms documentation for .NET*. Microsoft Learn. https://learn.microsoft.com/dotnet/desktop/winforms/

Microsoft. (2024c). *Microsoft.Data.SqlClient namespace*. Microsoft Learn. https://learn.microsoft.com/dotnet/api/microsoft.data.sqlclient

Microsoft. (2024d). *SQL Server documentation*. Microsoft Learn. https://learn.microsoft.com/sql/sql-server/

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.